DG INTERNS HUB | CYBERSECURITY INTERNSHIP

WEEK 09

SOC Introduction and Setup

Security Operations Center Fundamentals and Snort IDS Implementation

Technical report	Practical environment
SOC fundamentals, lab preparation, Snort IDS configuration and controlled detection tests.	VMware Workstation Ubuntu Server 24.04.5 LTS Snort 2.9.20

Prepared for internship assessment

Evidence-based documentation | 05 November 2026

Abstract

This report documents Week 09 learning and practical work focused on Security Operations Center (SOC) fundamentals and a local Snort Intrusion Detection System (IDS) lab. The supplied project archive contains SOC learning notes, an observations file and 11 screenshots. The evidence records Ubuntu Server 24.04.5 LTS in a VMware virtual machine, network interface details, Snort 2.9.20 installation and version output, successful Snort configuration validation, two custom rules, and ICMP/HTTP traffic exercises. The practical demonstrates signature matching and alert generation in an IDS context; it does not demonstrate inline prevention or traffic blocking.

Evidence and scope note

The narrative distinguishes direct screenshot evidence from statements in the project's findings file. The archive does not include a separate exported snort.conf/local.rules text file or raw packet capture. Rule text is transcribed from the supplied terminal screenshot. The HTTP exercise screenshot shows a plain-HTTP page rendered as source/markup in a terminal-like browser view; the companion findings note reports corresponding Snort alerts. Exact command lines and alert counts are not inferred where the screenshots do not make them legible.

Contents

Section	Page
1. SOC fundamentals and responsibilities	3
	4
3. SIEM, IDS and IPS	5
	6
5. Ubuntu Server setup	7
	8
7. Configuration validation	9
	10
9. Traffic tests and alert evidence	11–12
	13
11. Learning outcomes, conclusion and references	14

1. Introduction to Security Operations Centers

A Security Operations Center is an operational function that brings people, repeatable processes and security technologies together to monitor an organization's digital environment. A SOC collects telemetry, identifies patterns that may indicate threats, investigates alerts and coordinates a response. Its value is not simply the volume of alerts produced, but the quality and timeliness of decisions made from those signals.

Why a SOC is needed

Modern environments generate events across endpoints, servers, networks, identity platforms, cloud services and applications. Continuous monitoring helps identify suspicious activity outside normal business hours. Standardized triage and documentation make investigations more consistent, support escalation and preserve a record of decisions. Detection speed and accuracy depend on useful telemetry, tuned detections and clear ownership.

Core objectives and responsibilities

Responsibility	Operational purpose
Continuous monitoring	Review telemetry, dashboards and alerts across systems.
Threat detection	Apply signatures, analytics and correlation to identify suspicious patterns.
Alert triage	Validate context, prioritize and separate benign matches from actionable events.
Investigation	Correlate host, identity, application and network evidence to determine scope.
Incident response	Coordinate containment, eradication and recovery for confirmed incidents.
Threat intelligence	Use contextual knowledge to improve prioritization and detection logic.
Reporting	Record evidence, decisions, timelines, impact and lessons learned.

The Week 09 lab provides a small-scale example of one SOC technology function: a network IDS observes traffic and creates alerts that an analyst would then validate. Snort does not replace the analyst's contextual investigation.

2. SOC Analyst Levels and Monitoring Workflow

The L1/L2/L3 tier model is a common way to distribute alert-handling responsibilities. Exact boundaries vary by organization, staffing and playbooks; the tiers are a useful learning model rather than a universal standard.

Tier	Primary focus	Typical activities
L1 – Monitoring & triage	Initial alert handling	Monitor queues, validate basic context, collect timestamps/hosts/indicators, document and escalate.
L2 – Investigation & response	Detailed investigation	Correlate telemetry, assess scope and impact, investigate network/endpoint evidence, coordinate containment.
L3 – Advanced analysis	Complex cases and improvement	Threat hunting, malware/attacker-technique analysis, root-cause analysis, detection engineering and tuning.

SOC monitoring and incident-response workflow

Stage	What happens
1. Data collection	Logs, network events and endpoint/application telemetry are gathered.
2. Monitoring & detection	Rules and analytics identify activity that merits attention.
3. Alert triage	An analyst checks source, destination, time, asset criticality and rule context.
4. Investigation	Related events are correlated to determine whether activity is expected, suspicious or confirmed.
5. Response	For a confirmed incident, authorized teams contain, eradicate and recover.
6. Documentation	Evidence, actions, decisions and lessons learned are recorded; detections are refined.

In this lab, a Snort alert is an input to triage—not a final incident verdict. A benign ping or routine HTTP request can match a rule by design. An analyst must check whether the traffic was expected and whether other evidence changes its significance.

3. SIEM vs IDS vs IPS

These technologies are complementary but not interchangeable. An IDS inspects activity and reports matches; an IPS is positioned to enforce prevention; a SIEM aggregates and correlates events from multiple sources to support broader analysis.

Capability	SIEM	IDS	IPS
Primary role	Collect, normalize, correlate and analyze events	Detect patterns in network/host activity and alert	Detect and take configured preventive action
Typical input	Logs, events, alerts and telemetry	Packets or host activity	Traffic in an inline path or host enforcement point
Output	Searches, dashboards, correlated alerts and cases	Alerts / event records	Alerts plus actions such as drop or reject
Blocking	Not inherently; response may be orchestrated separately	Normally passive; no blocking in this lab	Can block/drop when configured inline
SOC use	Cross-source visibility and case context	Detection sensor and alert source	Prevention control

How they work together

Network and endpoint sensors produce telemetry. IDS alerts and system/application logs can be forwarded to a SIEM, where correlation can link events across assets and time. Analysts then investigate and initiate response through documented procedures. An IPS may block traffic inline, but its actions should also be logged and reviewed.

Project classification

The supplied evidence shows Snort operating as an IDS for signature-based detection and console alerting. There is no evidence of inline deployment, drop rules, enforcement mode or blocked packets. Therefore, this report does not claim IPS prevention.

4. Project Objectives and Lab Architecture

Project objectives

- Establish a Linux server virtual machine for foundational SOC monitoring practice.
- Configure the server identity and confirm network-interface state.
- Install Snort 2.9.20 and verify the installed version.
- Validate the Snort configuration with its test mode.
- Define custom ICMP and HTTP alert signatures using local rule identifiers 1000001 and 1000002.
- Generate controlled traffic and observe corresponding IDS alerts.

Lab inventory

Component	Evidence-backed value
Virtualisation	VMware Workstation (visible VM console context)
Operating system	Ubuntu Server 24.04.5 LTS; release 24.04, codename noble
Hostname	soc-server (hostname output and shell prompt)
Network interface	ens33; state UP in `ip a` output
IPv4 address	192.168.38.128/24 (observed in screenshot)
Default gateway	192.168.38.2 (observed in route output)
IDS	Snort 2.9.20 GRE, Build 82
Rule file	/etc/snort/rules/local.rules (shown in terminal)

Network note: The archive’s technical brief mentions VMware NAT (VMnet8), while the interface screenshot records the address and route above. The screenshot does not itself display the VMware virtual-network editor or explicitly label VMnet8. The NAT/VMnet8 description is therefore treated as supplied project context, while the IP/subnet values are transcribed from the screenshot. The displayed address is not replaced with an assumed default NAT subnet.

5. Ubuntu Server Installation and Configuration

The Ubuntu environment is the host platform for the IDS exercise. A consistent server identity and functioning network interface are prerequisites for repeatable monitoring and package installation.

Installation and operating-system identification



Figure 1. *Ubuntu release identification using lsb_release output. The screenshot identifies Ubuntu 24.04.5 LTS (release 24.04, noble).*

System updates

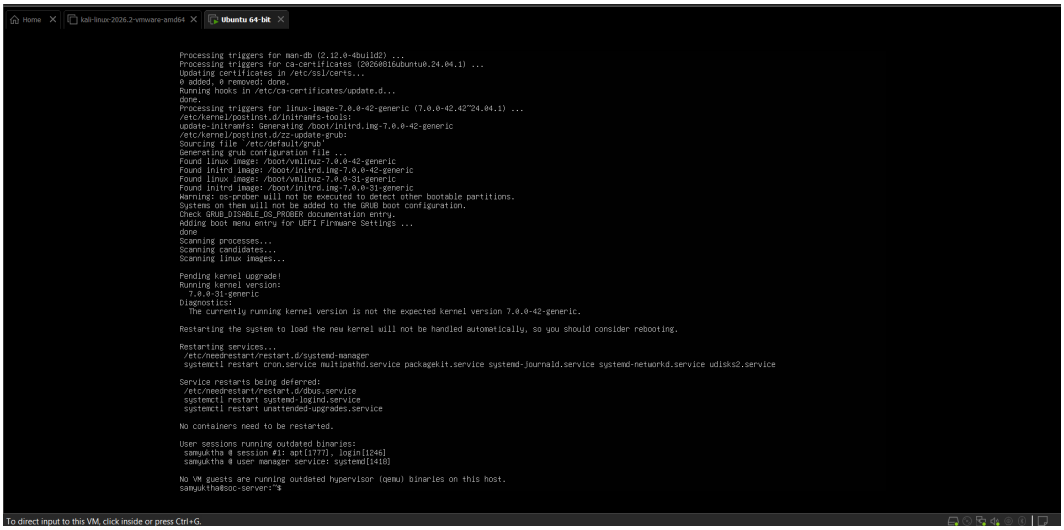


Figure 2. *Package update/upgrade activity and system maintenance output. The screenshot notes a pending kernel upgrade and recommends a reboot to load the new kernel; it does not establish that a reboot occurred.*

Hostname and network configuration



Figure 3. *Hostname/system information output identifying soc-server and Ubuntu Server 24.04.5 LTS.*

6. Network Configuration and Snort Installation

Network interface inspection

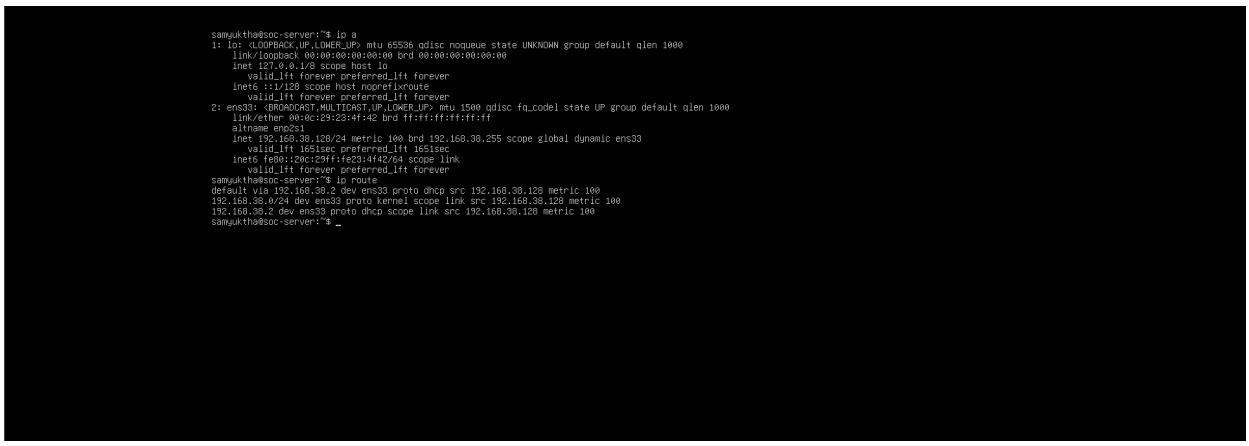


Figure 4. *`ip a` and route output: ens33 is UP with IPv4 192.168.38.128/24; the default route is via 192.168.38.2.*

The interface state and route provide evidence that the guest has a configured IPv4 interface and a default gateway. The screenshot alone does not prove end-to-end Internet reachability, DNS resolution or connectivity to a particular service; those require separate tests.

Snort installation

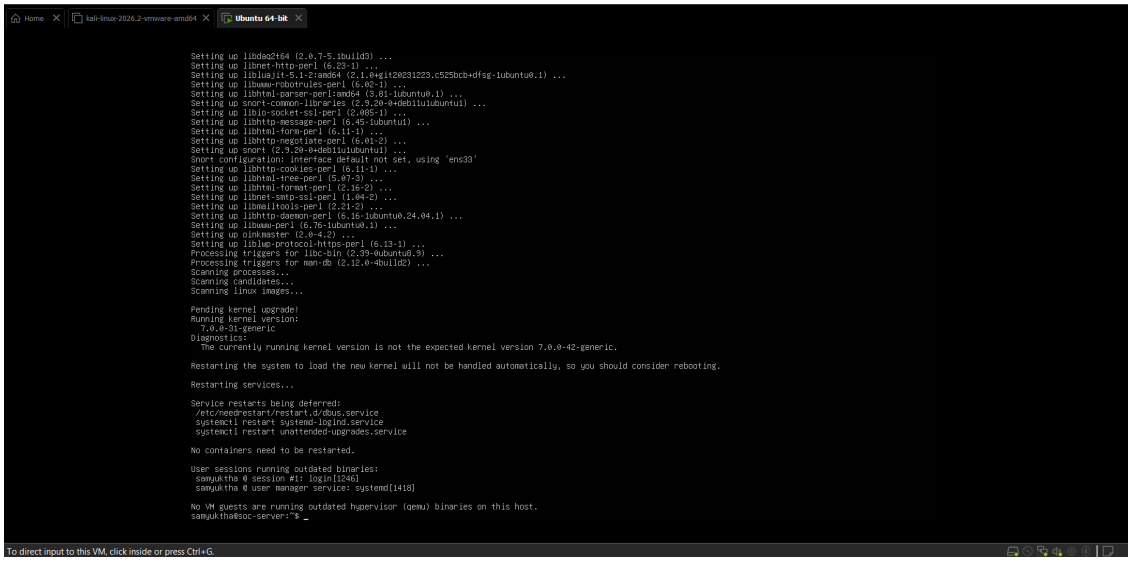


Figure 5. *Package setup output shows Snort 2.9.20 packages being configured and identifies ens33 as the default interface in the package configuration message.*

Snort is a signature-based network intrusion detection/prevention engine. In this project it is used for passive detection and alert generation. Installation output confirms package setup, while the separate version and configuration-test screenshots provide stronger verification of the executable and configuration state.

7. Snort Version Verification and Configuration Validation

Version verification



Figure 6. *`snort -V` output reports Snort 2.9.20 GRE (Build 82), together with library information.*

Version verification establishes which Snort build is available to the lab. It is important for reproducibility because rule syntax, options and behavior can differ between releases.

Configuration validation

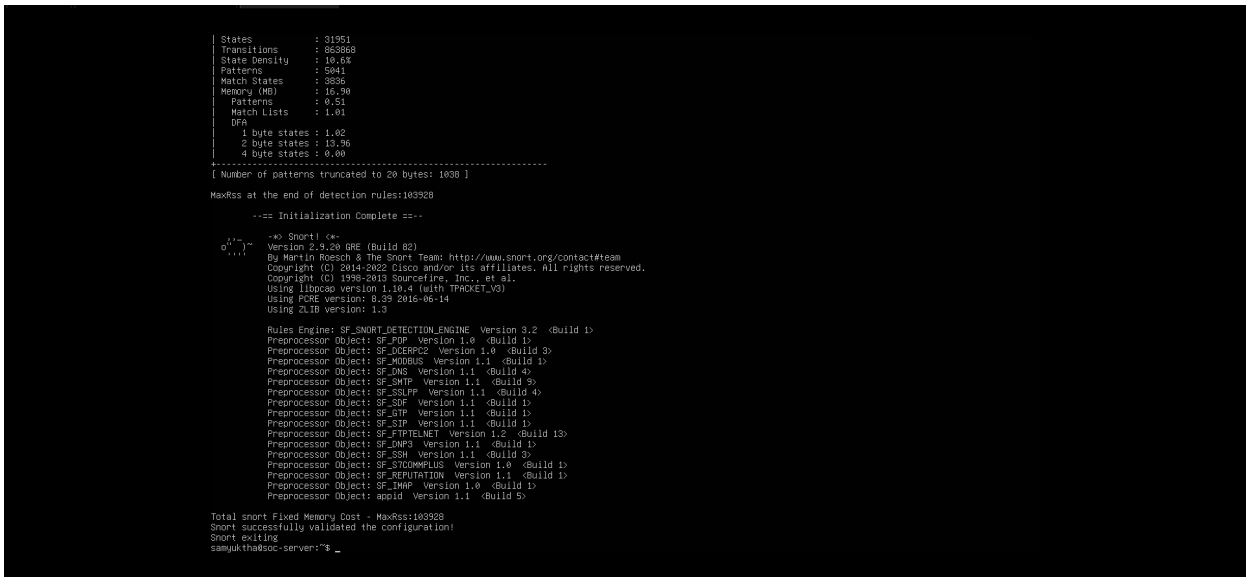


Figure 7. *Snort test-mode output ends with “Snort successfully validated the configuration!” and then exits.*

The project evidence shows a successful configuration test (the supplied technical brief identifies this as `snort -T`). This validates that Snort can initialize its configuration and load the relevant rule/preprocessor components at test time. It does not prove that the sensor is continuously running, that the correct interface is monitored in every invocation, or that all possible traffic is visible.

Operational distinction

A successful syntax/configuration test is a prerequisite check. Runtime validation additionally requires launching Snort on the intended interface, generating traffic that traverses that interface, and confirming that the expected rule alerts are produced. The archive includes alert screenshots for ICMP and HTTP exercises, supporting that next stage at a basic level.

8. Custom Snort Rules – ICMP and HTTP

The local rules screenshot displays two alert rules in `/etc/snort/rules/local.rules`. The rule messages and SIDs are directly visible. The rule expressions are transcribed below from the evidence.

ICMP detection rule

```
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; sid:1000001; rev:1;)
```

This rule alerts on ICMP traffic matching the direction and address variables in the rule header. `alert` requests an alert action; `icmp` selects the protocol; `any any` represents any source address and source port field; `$HOME_NET any` constrains the destination to the configured home network with any destination port field. SID 1000001 uniquely identifies the local signature in this ruleset; revision 1 marks its current revision.

HTTP traffic detection rule

```
alert tcp any any -> $HOME_NET 80 (msg:"HTTP Traffic Detected"; sid:1000002; rev:1;)
```

This rule alerts on TCP traffic directed to destination port 80 within `$HOME_NET`. It is a broad port-based match rather than a deep inspection of HTTP methods, URI paths or payload content. It can therefore match TCP traffic to port 80 without proving that the request is malicious or even that a complete HTTP transaction occurred.

Rule evidence

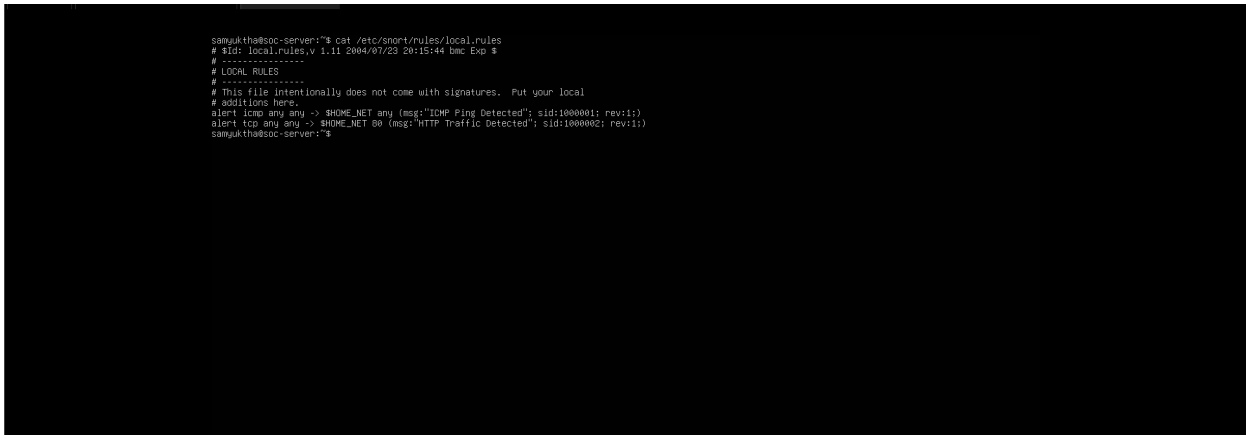


Figure 8. *The terminal shows both local.rules entries, their messages, SIDs 1000001/1000002 and revision values.*

Rule tuning considerations

The use of `$HOME_NET` depends on the variable being correctly defined in `snort.conf`. The archive does not include the full configuration file, so its exact network-variable definition cannot be independently inspected. In production, narrow source/destination scope, add meaningful metadata and suppress expected noisy traffic only after validating business requirements.

9. Traffic Generation and ICMP Detection

The project notes state that ICMP traffic was generated using ping and corresponding Snort alerts were observed. The screenshots provide complementary evidence: the local ICMP rule is visible in the rules file, and the traffic-monitoring capture shows repeated ICMP Ping Detected alerts.



Figure 9. *The local.rules terminal view shows the ICMP alert signature and SID 1000001.*

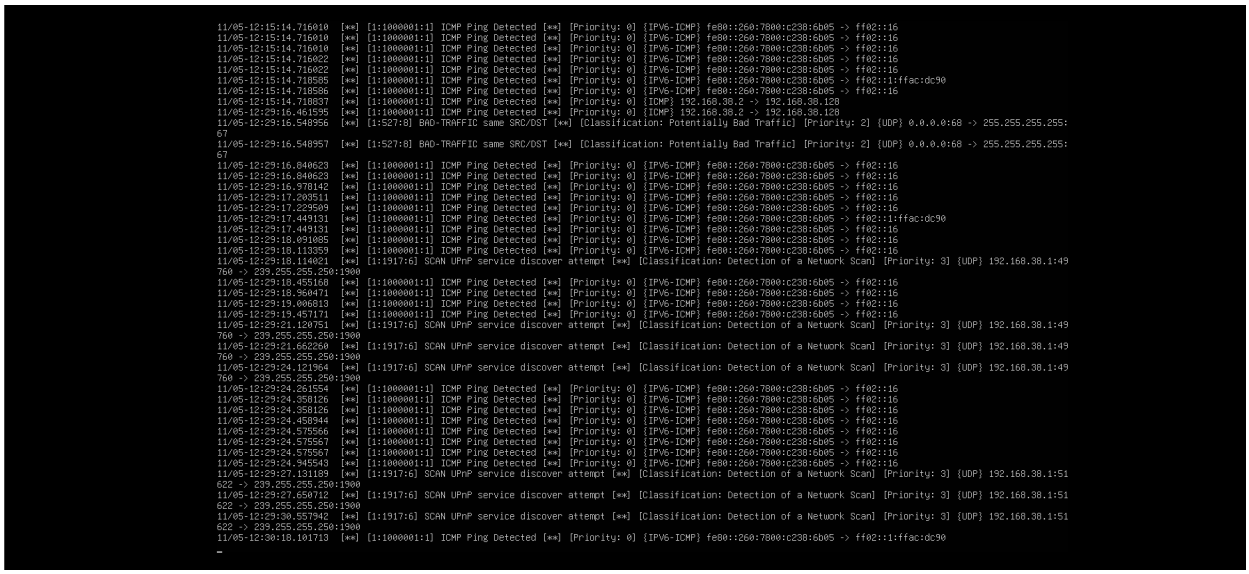


Figure 10. *Snort console output during the ping exercise includes repeated “ICMP Ping Detected” alerts. The same console also displays other built-in alerts, including network-scan-related signatures.*

Interpretation

The custom alert indicates that packets matched the configured ICMP signature. It is consistent with the intended controlled ping test. The presence of additional built-in alerts demonstrates that Snort may evaluate traffic against more than the two local rules. Those additional matches should be triaged individually; their appearance does not establish that an attack occurred. The screenshot is a console capture, not a packet capture or a complete event export, so exact counts and full five-tuple details are not asserted.

10. HTTP Traffic Test and Detection

The findings note records an HTTP traffic exercise using curl and states that HTTP detection alerts were verified. The supplied HTTP test screenshot shows the content of the NeverSSL page, including its explanation that the site uses unencrypted HTTP. The local rule screenshot separately confirms a TCP destination-port-80 alert rule with SID 1000002.

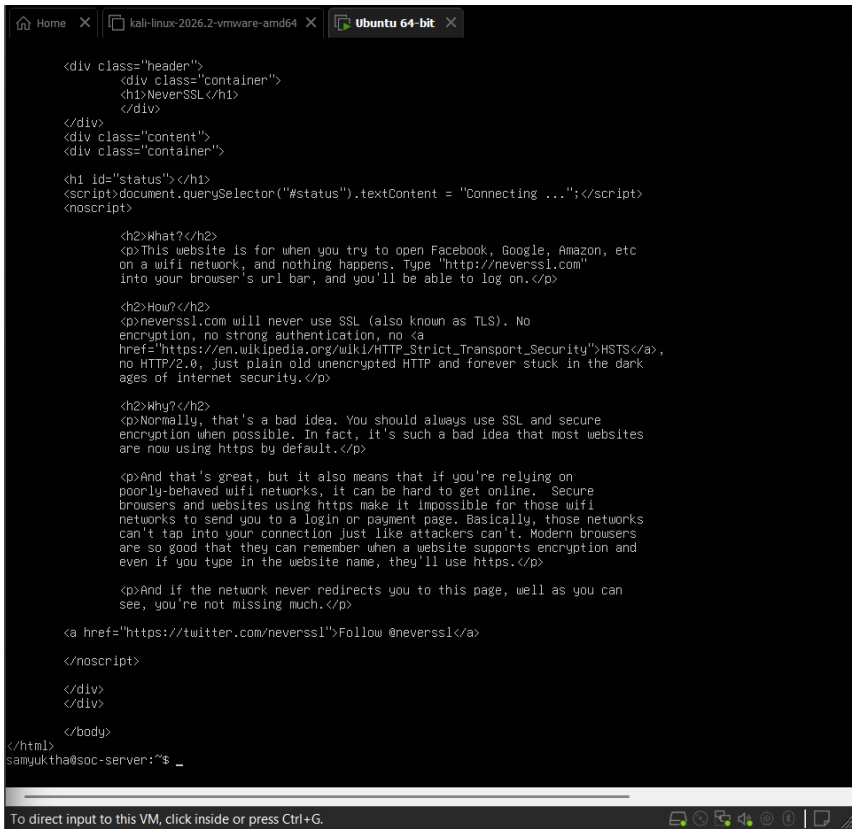


Figure 11. *HTTP test evidence: page content from neverssl.com is displayed. This supports the use of an HTTP endpoint for the exercise; it does not by itself show a Snort alert.*

Rule and alert relationship

The configured rule is `alert tcp any any -> $HOME_NET 80` and uses the message “HTTP Traffic Detected”. It detects matching TCP traffic directed to port 80 in the configured home network. The findings document reports corresponding Snort alerts, but the archive does not contain a clearly legible standalone screenshot of the HTTP alert console output. Accordingly, the report records HTTP detection as documented in the findings file, while identifying the missing direct alert screenshot as an evidence limitation.

Security interpretation

HTTP is unencrypted at the application layer, so content may be observable to an on-path observer. That property makes plain HTTP useful for a controlled demonstration, but it is not inherently evidence of malicious behavior. A production environment should prefer HTTPS and use appropriate TLS visibility controls where inspection is authorized and necessary.

11. Findings, Challenges and Recommendations

Evidence-based findings

ID	Finding / observation	Evidence status
F-01	Ubuntu Server 24.04.5 LTS environment and soc-server hostname are visible.	Direct screenshot
F-02	ens33 is UP; 192.168.38.128/24 and default gateway 192.168.38.2 are visible.	Direct screenshot
F-03	Snort 2.9.20 GRE Build 82 is installed and version output is shown.	Direct screenshot
F-04	Snort configuration test reports successful validation.	Direct screenshot
F-05	Custom ICMP rule SID 1000001 and HTTP rule SID 1000002 are visible in local.rules.	Direct screenshot
F-06	Ping test console shows repeated custom ICMP alerts and other built-in alerts.	Direct screenshot
F-07	HTTP exercise is documented; HTTP test page and rule are visible. A dedicated HTTP alert-console screenshot is not present.	Documented finding + partial direct evidence

Challenges and observations

• The update screenshot reports a pending kernel upgrade and suggests a reboot; no post-reboot verification is included.
• The supplied project brief names VMware NAT/VMnet8, but the screenshot only shows guest interface and route details; the virtual network editor configuration is not included.
• Full `snort.conf` and raw `local.rules` files are absent, limiting independent review of HOME_NET, interface and include directives.
• ICMP console output includes built-in scan-related alerts; each requires context-based triage.
• HTTP alert output is reported in the findings note but is not directly captured in a dedicated alert screenshot.

Recommendations and next steps

1. Preserve the exact Snort command line, interface selection and configuration path in a lab log.
2. Export `snort.conf`, `local.rules` and a sanitized Snort alert log alongside screenshots.
3. Capture a dedicated HTTP alert-console screenshot with timestamps and the matching SID 1000002 visible.
4. Record test source/destination, protocol, ports, timestamps and expected result for each controlled test.
5. Review HOME_NET and interface settings; document VMware NAT configuration explicitly.
6. Tune local rules to reduce noise and test both positive and negative cases.
7. Consider forwarding IDS alerts to a SIEM and documenting an L1 triage workflow.

12. Learning Outcomes and Conclusion

Learning outcomes

The work demonstrates foundational understanding of the SOC as a people/process/technology function; the distinction between L1 triage, L2 investigation and L3 advanced analysis; the complementary roles of SIEM, IDS and IPS; and the use of Snort signatures to alert on controlled network traffic. Practically, the evidence shows an Ubuntu Server environment, Snort installation and version verification, successful configuration validation, custom rule definitions and an ICMP alert exercise. The HTTP exercise is documented and supported by endpoint/rule evidence, with the direct alert capture gap noted above.

Conclusion

Week 09 established a basic SOC learning environment and demonstrated Snort IDS signature matching. The project's strongest directly evidenced outcomes are Snort 2.9.20 installation, successful configuration validation, visibility of both local rules and observed ICMP alerts during a ping exercise. The HTTP test is described as successful in the findings document, but a dedicated alert-console image would strengthen the evidence chain. These results show detection and alert generation only. They do not demonstrate traffic blocking, inline IPS operation, incident confirmation or production-grade SOC monitoring.

References

1. DG Interns Hub, *SOC Fundamentals: People, Process & Security Technologies*, Week 09 learning notes supplied in the project archive (Markdown and PDF).
2. Snort, *Snort 2.9 User Manual* and official documentation, Cisco/Snort project documentation: https://www.snort.org/documents
3. Ubuntu, *Ubuntu Server documentation*: https://ubuntu.com/server/docs
4. NIST, *Computer Security Incident Handling Guide*, SP 800-61 Rev. 2: https://doi.org/10.6028/NIST.SP.800-61r2

Appendix – Evidence inventory

The report incorporates the 11 PNG screenshots supplied in the archive: Ubuntu installation/release identification; system update; hostname; network interface/routes; Snort installation; Snort version; configuration test; ICMP rule; HTTP rule; ping/ICMP alert console; and HTTP test page. The archive also contains the SOC fundamentals notes and a findings-and-observations Markdown file. No separate raw configuration exports or packet captures were supplied.

Report prepared from the supplied Week 09 project archive | DG Interns Hub